

```
title: "「def train_dqn(env=None): if env is None: env = TradingEnv(train_df)」
exported: 2026-06-08 15:31:44
url: https://gemini.google.com/app/716ef0f3f3f1cade?hl=zh-TW
messages: 4
```

## 「def train\_dqn(env=None):

```
if env is None:
    env = TradingEnv(train_df)
```

### You

有Target Network 的DQN 是double DQN 嗎？

### Gemini

答案是：不是喔！這是一個非常經典且常見的誤解。

簡單來說：標準的 DQN 本身就已經包含了 Target Network（目標網路）。

有 Target Network 只是標準 DQN 的基本配備，而 Double DQN（DDQN）則是進一步改變了利用這兩個網路計算目標值的方式。

下面幫你釐清這兩者的關鍵差異：

## 1. 為什麼標準 DQN 就有 Target Network？

在最早的 DQN（Nature 2015 版本）中，為了讓訓練穩定，就已經引入了兩個結構一模一樣的網路：

- **Current Network**（當前網路，參數為  $\theta$ ）：負責輸出行為、即時更新參數。
- **Target Network**（目標網路，參數為  $\theta^-$ ）：隔一段時間才從 Current Network 複製一次參數，用來提供穩定的目標值。

如果不加 Target Network，我們更新網路時，目標值（Target）會隨著每次更新一起變動，這就像「射箭時靶自己會一邊移動」一樣，網路極難收斂。

在標準 DQN 中，計算目標值（TD Target）的公式如下：

$$Y_t^{DQN} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta^-)$$

這裡的問題在於：選擇「哪一個動作能帶來最大價值」以及「評估這個動作價值多少」，全部都由 **Target Network ( $\theta^-$ )** 一手包辦。這會導致  $\max$  操作很容易把雜訊帶來的過高估計（Overestimation）給繼承下去。

## 2. 什麼才是 Double DQN（DDQN）？

Double DQN（2015 年底提出）是為了解決上述 DQN 的過度估計（Overestimation Bias）問題。

Double DQN 沒有增加新的網路（它依然是使用原本 DQN 就有的 Current Network 和 Target Network），但它把「選擇動作」和「評估動作」這兩件事拆開給不同的網路做：

1. 由 **Current Network ( $\theta$ )** 決定動作：看在下一個狀態  $S_{t+1}$  下，哪個動作的  $Q$  值最大。
2. 由 **Target Network ( $\theta^-$ )** 評估價值：算出該動作在目標網路中的實際  $Q$  值。

它的目標值計算公式變成了：

$$Y_t^{DDQN} = R_{t+1} + \gamma Q\left(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta); \theta^-\right)$$

## 核心差異總結

可以用一個簡單的比喻來理解：

- **標準 DQN**：球探（Target Network）自己挑選了他認為最棒的球員，然後自己給這個球員打分数（容易流於主觀、過度自我感覺良好）。
- **Double DQN**：教練（Current Network）負責挑選他覺得最棒的球員，但分数必須由公正的第三方球探（Target Network）來打（分工合作，避免盲目樂觀）。

所以，有 Target Network 只是 DQN，懂得「用 Current Network 選動作、用 Target Network 算分数」的才是 Double DQN。

### You

```
「def train_dqn(env=None): if env is None: env = TradingEnv(train_df)」 這代表我輸入任何env都會變TradingEnv(train_df)嗎？
```

### Gemini

答案是：不會喔！剛好相反，這段程式碼的意思是「只有在你『沒有』輸入任何 env 時，它才會幫你自動預設為 TradingEnv(train\_df)」。

如果你自己有傳入一個特定的 env，它就會直接使用你傳入的那一個，絕對不會被覆蓋。

我們可以把這段程式碼拆解成兩個步驟來看它的運作邏輯：

#### 1. 預設值設定 (env=None)

```
def train_dqn(env=None):
```

這行的 `env=None` 代表 `env` 是一個\*\*選填 (Optional)\*\* 的參數。如果你呼叫函數時括號裡面什麼都不放，`env` 在進去函數內部時，預設就會是 `None`。

## 2. 條件判斷 (if env is None:)

```
if env is None:
    env = TradingEnv(train_df)
```

這裡就像一個守門員。只有當 `env` 的值真的是 `None` 的時候，才會觸發裡面的縮排程式碼，把 `env` 指定為 `TradingEnv(train_df)`。

---

### 💡 實際執行的兩種情況

- 情況一：你偷懶什麼都沒傳

```
train_dqn()
# 運作流程：env 預設為 None -> 滿足「if env is None」-> env 被設定為 TradingEnv(train_df)
```

- 情況二：你有自己指定別的環境

```
my_custom_env = GymStockEnv()
train_dqn(env=my_custom_env)
# 運作流程：env 現在是 my_custom_env -> 不滿足「if env is None」（因為它不是 None）-> 直接跳過 if 區塊 -> env 依然是你的 my_custom_e
````
```

所以這是一個在 Python 中非常常見的\*\*「預設機制」\*\*。它讓你平常可以少打幾個字（直接用預設的 `TradingEnv`），但同時又保留了讓你未來可以自由傳入其他環境。